

Proyecto 1

Fundamentos de sistemas Computacionales

Grupo 02

Detrás de una caja: Creacion de un juego “Whack-a-Mole”

Por: Alvaro Granados Fernandez

Introducción:

La creación y formulación de este proyecto se hizo con el afán de estimular y poner en práctica los conocimientos anteriormente adquiridos en los talleres y clases del curso “Fundamentos de Sistemas Computacionales”. El objetivo principal de este trabajo es la comprensión y aplicación de los componentes físicos (hardware) y un lenguaje de programación anteriormente estudiado que es Python (software), para la creación de una maquina básica que fusiona como los populares juegos “Whack-a-Mole”.

En este proyecto, se utilizarán botones y swiches para la comunicación entre la maquina el usuario, al igual que leds y segmentos para ofrecer información relevante al jugador. Estos compuestos permitirán la interacción entre el usuario y el microcontrolador Raspberry Pi Pico W, el cual sera altamente utilizado a lo largo de este proyecto.

Con la información previamente suministrada, procederemos a analizar cómo se utilizaron los componentes anteriormente mencionados junto con el lenguaje de programación Python para poder lograr el objetivo de crear un sistema de interacción de respuesta rápida (juego de golpear topes).

Desarrollo:

1. Variables iniciales y

bibliotecas: Al inicio del código, se escriben las variables que serán usadas para enviar o recibir información entre el jugador y la máquina. También se inician las bibliotecas a utilizar, al igual que los elementos de estas que se usaran.

```
def apagarLeds(lista):  
    for led in lista:  
        led.value(0)
```

```
def numSegmento(n):  
    if n == 0:  
        apagarLeds(segLeds)  
        segA.value(1)  
        segB.value(1)  
        segC.value(1)  
        segD.value(1)  
        segE.value(1)  
        segF.value(1)
```

```
import time  
import random  
from machine import Pin, PWM
```

```
#----- Miselaneos -----  
  
buzzer = PWM(Pin(10))  
switch_1 = Pin(8, Pin.IN)  
switch_2 = Pin(5, Pin.IN)  
  
#----- Botones -----  
  
boton_1 = Pin(11, Pin.IN)  
boton_2 = Pin(12, Pin.IN)  
boton_3 = Pin(13, Pin.IN)  
  
#----- Leds -----  
  
boton_led = Pin(14, Pin.OUT)  
led_1 = Pin(15, Pin.OUT)  
led_2 = Pin(16, Pin.OUT)  
led_3 = Pin(17, Pin.OUT)  
  
#----- Led Segmentos -----  
  
segA = Pin(18, Pin.OUT)  
segB = Pin(19, Pin.OUT)  
Pin(20, Pin.OUT)  
Pin(21, Pin.OUT)  
Pin(22, Pin.OUT)
```

2. Funciones led y selectores: En el código, se utilizan diferentes funciones que orquestan el encendido y apagado de los leds en un orden específico, con la intención de comunicar un mensaje o mejorar la estética. Por ejemplo, la función presente en la imagen (numSegmento(n)) activa los leds del segmento con el objetivo de dar a conocer un número al usuario, siendo este la dificultad, puntuación, u otros. El resto de los leds utilizados cuentan con sus propias funciones,

pero a comparación del segmento, su código solo apaga dichos leds al mismo tiempo. Luego, se tienen las funciones **selectoras**, las cuales toman el numero ofrecido por el fabricante y devuelven el led o el boton que coincida con el numero previamente dado. A pesar de ser funciones simples, estas permiten la manipulación de su encendido y apagado (al igual que su elección para recibir una señal) meramente con el uso de números enteros, ofreciendo mayor control sobre cuales circunstancias serán los leds o los botones tomados en cuenta.

3.Musica: Para la música, se utiliza una función que toma un número y el tempo. El numero dado por el fabricante selecciona una de las 5 "partituras" almacenadas en la función. El tempo (al igual que su significado en la música), rige la duración de cada canción, más específicos: la duración de cada una de las notas. Esta toma las "partituras" (listas), y a través de un comando "for" las reproduce por un tiempo determinado. La funcion transforma los segundos dados a travez de la variable tempo y los convierte en milisegundos para poder utilizar las funciones de la biblioteca "time".

```
def cajaMusical(numero, tempo):
    tonos = {
        "C4": 261,
        "C#4": 277,
        "D4": 294,
        "D#4": 311,
        "D#_4": 320,
        "E4": 330,
        "F4": 349,
        "F#4": 370,
        "G4": 392,
        "G#4": 415,
        "A4": 440,
        "A#4": 466,
        "A#_4": 480,
        "B4": 494,
        "B#4": 508,
        "C5": 523
    }
```

4.Bucle "while": Esta sección se dividirá en diferentes partes, siendo que el bucle principal almacena la mayoría de los comportamientos condiciones que hacen el funcionamiento del código una posibilidad:

```
while True:

    numSegmento(dificultad)
    musica = random.randint(1, 3)

    while n < 1:
        cajaMusical(musica, 0.15) #cambiar 4
        n +=1
    buzzer.duty_u16(0)

    #Selector de dificultad por permisos

    boton_led.value(1)

    if switch_1.value() == 1 and permisos.count(2) != 0:
        dificultad = 2
        led_1.value(1)
        led_2.value(1)
        led_3.value(0)
```

4,1.Pantalla de selección: En esta parte del código localizada al inicio del bucle, se localiza la selección de dificultades. Antes de que el sistema permita al usuario seleccionar la dificultad a jugar, se reproducirá momentáneamente una de las canciones previamente guardadas,

seleccionadas aleatoriamente en cada juego. Posteriormente, se le permitirá seleccionar la dificultad con la activación de los 2 switches integrados, siempre y cuando este tenga el acceso a esas dificultades. Este acceso es una lista fuera de la iteración, que se actualiza cada vez que se completa una dificultad si errar. Una vez completada la dificultad necesaria, se le permitira avanzar al siguiente nivel. De no ser seleccionada ninguna dificultad, o se intenta utilizar los switches sin haber pasado la dificultad previa, se seleccionará la dificultad 1 por defecto. Estos permisos se verán eliminados una vez se desconecte el sistema, o se reinicie el código desde la aplicación utilizada.

```

if boton_2.value() == 1:
    boton_led.value(0)
    interruptorInf(conglomeradoLeds,1)
    time.sleep(0.3)
    interruptorInf(conglomeradoLeds,0)
    time.sleep(0.7)

topos = 5 if dificultad == 1 else (7 if dificultad == 2 else 4)
duracionPatron = 1000 if dificultad == 1 else (500 if dificultad == 2 else 200)

strikes = []

inicioCiclo = time.ticks_ms()

```

4.2. Inicio del juego: El juego comenzara una vez se presione el botón central de la "consola" (este se encuentra iluminado para ser fácilmente

identificado). Con el uso de las **funciones led**, se apagan las luces para evitar alguna confusión por parte del usuario y se define la variable "topos" y "duracionPatron". Estas variables dictan la cantidad de rondas en las que se activaran los botones (número de "topos"), y cuánto tiempo se tendrá para presionar los botones y obtener puntos (la duración del patrón).

Una vez se tiene la duración de cada patrón y las rondas a jugar, se selecciona

aleatoriamente el botón que tendrá que ser presionado. De

encontrarse en la dificultad 3, se verificará que los 2 botones que serán presionados no sean los mismos. De igual manera, se encenderán

las luces de los botones que se deben presionar (el led que se encenderá se encuentra al frente del botón), pero solo se encenderá 1 de encontrarse en la dificultad 1 o 2.

```

for nCiclo in range(topos):
    tiempoActual = time.ticks_ms() #Guarda el inicio del ciclo

    nRandom2 = random.randint(1, 3)
    nRandom = random.randint(1, 3)
    if dificultad == 3 and nRandom == nRandom2:
        lista = [1, 2, 3]
        lista.remove(nRandom) # elimina el número duplicado
        nRandom2 = random.choice(lista)

    boton_p = selectorBoton(nRandom) #Elige un boton
    boton_p2 = selectorBoton(nRandom2)
    led_p = selectorLed(nRandom)
    led_p2 = selectorLed(nRandom2)
    led_p.value(1)
    if dificultad == 3:
        led_p2.value(1)

    rPositiva = False
    rNegativa = False
    inicioCiclo = time.ticks_ms()

```

```

while (time.time_diff(tiempoActual, inicioCiclo) != duracionPatron) and rPositiva == False:
    tiempoActual = time.time_ms()
    if dificultad != 3:

        if boton_p.value() == 1: #Si se presiona el boton
            led_p.value(0)

            print(puntos)
            cajaMusical(4, 0.1)
            buzzer.duty_u16(0)

            puntos += 1 #Sumar puntos, yay!
            strikes.append(1)
            rPositiva = True

    elif dificultad == 3:

```

4.3. Obtención de puntos: Para mantener la reproducción de un código por un tiempo en específico, se utilizó la función "while" junto al comando de la librería "time": "time.time_diff". Antes de iniciar la iteración de un código, se actualiza el valor de la variable "inicioCiclo" con el tiempo actual (no confundir con la variable "tiempoActual"). Posteriormente, se inicia la iteración y se actualiza constantemente la variable "tiempoActual" con (valga la redundancia) el tiempo actual. Con el comando "time_diff", se mide el tiempo de diferencia que hay entre ambas variables. Si el tiempo recorrido iguala la duración del patrón, se detiene la iteración. Dentro de la iteración, se activará el código encargado de la puntuación si se presiona el botón en el rango de tiempo ofrecido. Dicho código apagará el led del botón que debe ser presionado, reproducirá un sonido de obtención de punto, agregará un strike afirmativo, se actualizarán el número mostrado en el segmento con los puntos actuales, y **activará la "ruptura positiva"**. La "ruptura positiva" previene la obtención constante de puntos en una sola ronda, deteniendo la iteración antes de lo previsto. La "ruptura positiva" junto a la "ruptura negativa" se actualizan antes de iniciar la iteración en base a tiempo (el "while time_diff" para una mejor comprensión). Si la dificultad elegida fue la 3 (dificultad máxima), se usará la misma línea de código utilizada para el "boton_p", pero repetido y con el "boton_p2" en su lugar. Una vez se salga de la iteración en base a tiempo, se detiene el programa por 0.5 segundos con el uso de el comando "time.sleep()" para evitar que el jugador se aburra. Cabe resaltar que esta línea de código se encuentra dentro "for _ range(topos)", por lo que cada parte del código afecta individualmente a cada una de las interacciones entre los "topos" y el jugador.

```

if rPositiva == False and rNegativa == False: #Analiza si no se agrego un punto en el juego anterior
    strikes.append(0)
    cajaMusical(5, 0.1)
    buzzer.duty_u16(0)
    rNegativa = True
    print("strike")

if strikes.count(0) == 3: #Si se cumple la cantidad de strikes, se rompe la iteracion
    print("break")
    break

```

4.4. Sistema de reprencion: Las “rupturas” previamente mencionadas, permiten que el sistema comprenda si debe (o no) dar puntos o “strikes”. Debido a que la “ruptura positiva” solo se actualiza cuando se cambia de ronda, esta comunicará al sistema si es que se ganaron puntos. Si la “ruptura positiva” no fue afectada, no se ganaron puntos, lo que por ende amerita una reprensión. Antes de explicar el uso de la “ruptura negativa”, procederé a explicarle como se reprende al usuario. La lista “strikes” almacena los números 1 y 0: se agregará un 1 si se obtuvo un punto, o un 0 si no se acertó. Dicho cero es agregado en esta parte del código. Posterior a ello, se reproducirá un sonido fácilmente distinguido como “algo negativo”. Luego, se actualizará la “ruptura negativa” para evitar la adición innecesaria de strikes. Una vez haya suficientes “strikes” en la lista (el recuento de ceros es igual a 3), se rompe el ciclo “for _ range(topos)”, terminando el juego antes de lo previsto. Ambas rupturas son actualizadas a “false” al inicio de cada ciclo “for”.

```

if (puntos == 5 and dificultad == 1) or (puntos == 7 and dificultad == 2):
    if permisos.count(2) == 0:
        permisos.append(2)

    elif permisos.count(3) == 0 and permisos.count(2) == 1:
        permisos.append(3)

interruptorInf(conglomeradoLeds,0)

inicioCiclo = time.ticks_ms()
tiempoActual = time.ticks_ms()
while (time.ticks_diff(tiempoActual, inicioCiclo) < 3000):
    tiempoActual = time.ticks_ms()
    if metronomo== 1:
        interruptorInf(conglomeradoLeds,1)

        time.sleep(0.4)
        metronomo = 0
        numSegmento(puntos)
    else:
        interruptorInf(conglomeradoLeds,0)
        time.sleep(0.4)
        metronomo = 1
        apagarLeds(segLeds)
puntos = 0
dificultad = 1
n = 0

```

Una vez completado el juego, se toman los puntos obtenidos para deducir si el jugador gana acceso a las siguientes dificultades. Este paso se omite si ya se tiene permiso para dichas dificultades. Finalmente, se

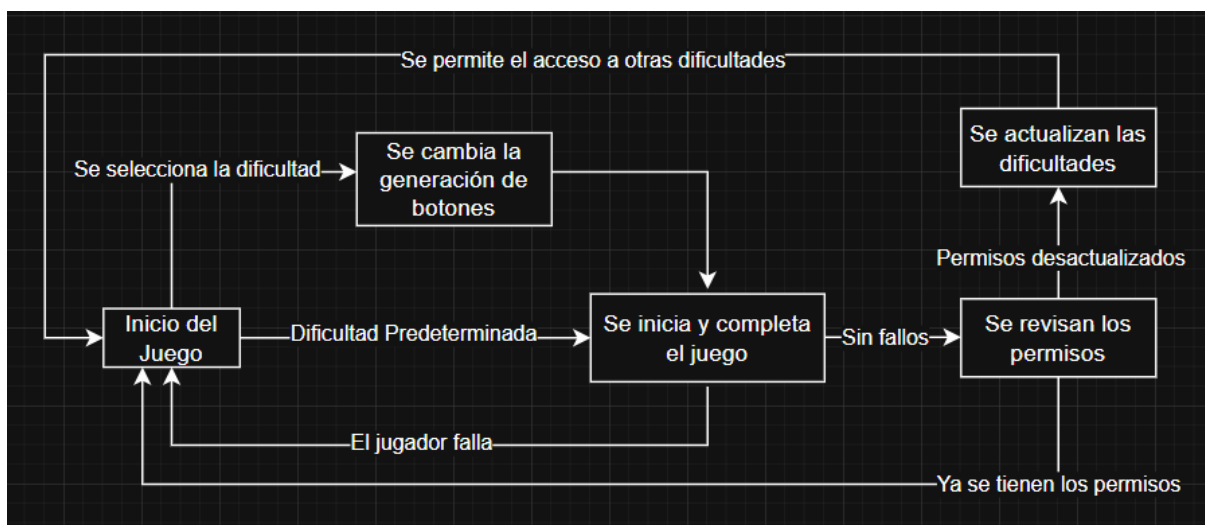
activa una pequeña animación de finalización, encendiendo y apagando los leds.

Marco teórico y resultados:

Debido a que ambas rubricas fueron parcialmente cubiertas en el desarrollo, se aclararán algunos sobre el código, al igual que se adjuntarán en orden las screenshots del código, y el diagrama de flujos:

- Los comandos “print()” solo se utilizaron para corroborar el funcionamiento adecuado del código.
- En este código, se utilizó la función “while” en base a tiempo en conjunto con el comando “sleep()” para evitar que los cambios fueran bruscos, permitiendo que el usuario seleccione cada botón a tiempo.
- La “invención” del “while en base a tiempo” se basó en la intencion de ejecutar una parte en específico del codigo por un tiempo determinado. Se intento utilizar “sleep()”, pero este resultado inefectivo.
- Se decidió utilizar un “while” principal para poder realizar el juego siempre que se haya completado un ciclo.
- Al igual que el “while” en base a tiempo, se acompañó al “while” principal con un “for” para cada ronda, permitiendo que se ejecute una lista de codigo en cada aparición de un “topo”.
- La variable como puntos, dificultad o n (música) se reinician al finalizar un ciclo del while principal, permitiendo un inicio “fresco” cada vez que se inicie el juego.

Diagrama de flujo:



Capturas de pantalla:


```
import time
import random
from machine import Pin, PWM
```

```
#----- Miselaneos -----
```

```
buzzer = PWM(Pin(10))
switch_1 = Pin(8, Pin.IN)
switch_2 = Pin(5, Pin.IN)
```

```
#----- Botones -----
```

```
boton_1 = Pin(11, Pin.IN)
boton_2 = Pin(12, Pin.IN)
boton_3 = Pin(13, Pin.IN)
```

```
#----- Leds -----
```

```
boton_led = Pin(14, Pin.OUT)
led_1 = Pin(15, Pin.OUT)
led_2 = Pin(16, Pin.OUT)
led_3 = Pin(17, Pin.OUT)
```

```
#----- Led Segmentos -----
```

```
segA = Pin(18, Pin.OUT)
segB = Pin(19, Pin.OUT)
segC = Pin(20, Pin.OUT)
segD = Pin(21, Pin.OUT)
segE = Pin(22, Pin.OUT)
segF = Pin(26, Pin.OUT)
segG = Pin(27, Pin.OUT)
segLeds = [segA, segB, segC, segD, segE, segF, segG]
```

```
def apagarLeds(lista):
    for led in lista:
        led.value(0)

def numSegmento(n):
    if n == 0:
        apagarLeds(segLeds)
        segA.value(1)
        segB.value(1)
        segC.value(1)
        segD.value(1)
        segE.value(1)
        segF.value(1)
        segG.value(1)

    elif n == 1:
        apagarLeds(segLeds)
        segB.value(1)
        segC.value(1)

    elif n == 2:
        apagarLeds(segLeds)
        segA.value(1)
        segB.value(1)
        segD.value(1)
        segE.value(1)
        segG.value(1)

    elif n == 3:
        apagarLeds(segLeds)
        segA.value(1)
        segB.value(1)
        segC.value(1)
        segD.value(1)
        segG.value(1)

    elif n == 4:
        apagarLeds(segLeds)
        segB.value(1)
        segC.value(1)
        segF.value(1)
        segG.value(1)

    elif n == 5:
        apagarLeds(segLeds)
        segA.value(1)
        segC.value(1)
        segD.value(1)
        segF.value(1)
        segG.value(1)

    elif n == 6:
        apagarLeds(segLeds)
        segA.value(1)
        segC.value(1)
        segD.value(1)
        segE.value(1)
        segF.value(1)
        segG.value(1)

    elif n == 7:
        apagarLeds(segLeds)
        segA.value(1)
        segB.value(1)
        segC.value(1)

    elif n == 8:
        apagarLeds(segLeds)
        segA.value(1)
        segB.value(1)
        segC.value(1)
        segD.value(1)
        segF.value(1)
        segG.value(1)

    elif n == 9:
        apagarLeds(segLeds)
        segA.value(1)
        segB.value(1)
        segC.value(1)
        segD.value(1)
        segF.value(1)
        segG.value(1)

    elif n == 10:#a
        apagarLeds(segLeds)
        segA.value(1)
        segB.value(1)
        segC.value(1)
        segD.value(1)
        segE.value(1)
        segF.value(1)
        segG.value(1)

    elif n == 11:#b
        apagarLeds(segLeds)
        segC.value(1)
        segD.value(1)
        segE.value(1)
        segF.value(1)
        segG.value(1)

    elif n == 12:#c
        apagarLeds(segLeds)
        segD.value(1)
        segE.value(1)
        segG.value(1)

    elif n == 13:#d
        apagarLeds(segLeds)
        segB.value(1)
        segC.value(1)
        segD.value(1)
        segE.value(1)
        segF.value(1)
        segG.value(1)

    elif n == 14:#e
        apagarLeds(segLeds)
        segA.value(1)
        segD.value(1)
        segE.value(1)
        segF.value(1)
        segG.value(1)

    elif n == 15:#f
        apagarLeds(segLeds)
        segA.value(1)
        segE.value(1)
        segF.value(1)
        segG.value(1)

#-----

def cajaMusical(numero, tempo):
    tonos = {
        "C4": 261,
        "C#4": 277,
        "D4": 294,
        "D#4": 311,
        "D#_4": 320,
        "E4": 330,
        "F4": 349,
        "F#4": 370,
        "G4": 392,
        "G#4": 415,
        "A4": 440,
        "A#4": 466,
        "A#_4": 480,
        "B4": 494,
        "B#4": 508,
        "C5": 523
    }
```

```

partitura = []
if numero == 1:
    partitura = ["C4", "E4", "G4", "G#4", "G#4", "G4", "C#4", "C4"]
elif numero == 2:
    partitura = ["D#4", "C4", "D#4", "C4", "D#4", "D#4", "D#_4", "C4", "C4"]
elif numero == 3:
    partitura = ["D#4", "C4", "E4", "E4", "F4", "F4", "D#4", "C4", "E4", "C4", "E4", "F4", "F4"]
elif numero == 4: #Acierto
    partitura = ["C4", "G4"]
elif numero == 5: #Strike
    partitura = ["A4", "D4"]

for nota in partitura:

    tiempoActual = time.time()
    inicioCiclo = time.time()

    while time.time() - inicioCiclo < tiempo*1000:
        buzzer.duty_u16(10000)
        buzzer.freq(tonos[nota])
        tiempoActual = time.time()
    buzzer.duty_u16(0)

def selectorBoton(n):
    if n == 1:
        return boton_1
    elif n == 2:
        return boton_2
    else:
        return boton_3

def selectorLed(n):
    if n == 1:
        return led_1
    elif n == 2:
        return led_2
    else:
        return led_3

def interruptorInf(lista, valor):
    for led in lista:
        led.value(valor)

conglomeradoLeds = [led_1, led_2, led_3]

#----- Iteracion y datos
dificultad = 1 #Dificultad, sera 1 si no se especifica
puntos = 0

n = 0

permisos = []

tonoGanador = []
tonoPerdedor = []

interruptorInf(conglomeradoLeds, 0)
boton_led.value(0)

```

```

while True:

    numSegmento(dificultad)
    musica = random.randint(1, 3)

    while n < 1:
        cajaMusical(musica, 0.15) #cambiar 4
        n +=1
    buzzer.duty_u16(0)

    #Selector de dificultad por permisos

    boton_led.value(1)

    if switch_1.value() == 1 and permisos.count(2) != 0:
        dificultad = 2
        led_1.value(1)
        led_2.value(1)
        led_3.value(0)

    elif switch_2.value() == 1 and permisos.count(3) != 0:
        dificultad = 3
        led_1.value(1)
        led_2.value(1)
        led_3.value(1)

    elif switch_1.value() == 0 and switch_2.value() == 0:
        dificultad = 1
        led_1.value(1)
        led_2.value(0)
        led_3.value(0)

    inicioCiclo = time.ticks_ms()

    if boton_2.value() == 1:

        boton_led.value(0)

        interruptorInf(conglomeradoLeds,1)

        time.sleep(0.3)

        interruptorInf(conglomeradoLeds,0)

        time.sleep(0.7)

        topos = 5 if dificultad == 1 else (7 if dificultad == 2 else 4)
        duracionPatron = 1000 if dificultad == 1 else (500 if dificultad == 2 else 200)

```

```

strikes = []

inicioCiclo = time.ticks_ms()

for nCiclo in range(topos):
    tiempoActual = time.ticks_ms() #Guarda el inicio del ciclo

    nRandom2 = random.randint(1, 3)
    nRandom = random.randint(1, 3)
    if dificultad == 3 and nRandom == nRandom2:
        lista = [1, 2, 3]
        lista.remove(nRandom) # elimina el número duplicado
        nRandom2 = random.choice(lista)

    boton_p = selectorBoton(nRandom) #Elige un boton
    boton_p2 = selectorBoton(nRandom2)
    led_p = selectorLed(nRandom)
    led_p2 = selectorLed(nRandom2)
    led_p.value(1)
    if dificultad == 3:
        led_p2.value(1)

    rPositiva = False
    rNegativa = False
    inicioCiclo = time.ticks_ms()
    while (time.ticks_diff(tiempoActual, inicioCiclo) != duracionPatron) and rPositiva == False:
        tiempoActual = time.ticks_ms()
        if dificultad != 3:

            if boton_p.value() == 1: #Si se presiona el boton
                led_p.value(0)

                print(puntos)
                cajaMusical(4, 0.1)
                buzzer.duty_u16(0)

                puntos += 1 #Sumar puntos, yay!
                strikes.append(1)
                rPositiva = True

        elif dificultad == 3:

            if boton_p.value() == 1: #Si se presiona el boton
                led_p.value(0)

                print(puntos)
                cajaMusical(4, 0.1)
                buzzer.duty_u16(0)

```

```

elif dificultad == 3:

    if boton_p.value() == 1: #Si se presiona el boton
        led_p.value(0)

        print(puntos)
        cajaMusical(4, 0.1)
        buzzer.duty_u16(0)

        puntos += 1 #Sumar puntos, yay!
        strikes.append(1)
        rPositiva = True

    elif boton_p2.value() == 1: #Si se presiona el boton
        led_p2.value(0)

        print(puntos)
        cajaMusical(4, 0.1)
        buzzer.duty_u16(0)

        puntos += 1 #Sumar puntos, yay!
        strikes.append(1)
        rPositiva = True

    numSegmento(puntos)
    led_p.value(0)

if dificultad == 3:
    led_p2.value(0)

time.sleep(0.5)

if rPositiva == False and rNegativa == False: #Analiza si no se agrego un punto en el juego anterior
    strikes.append(0)
    cajaMusical(5, 0.1)
    buzzer.duty_u16(0)
    rNegativa = True
    print("strike")

if strikes.count(0) == 3: #Si se cumple la cantidad de strikes, se rompe la iteracion
    print("break")
    break

```

```

        numSegmento(puntos)
        led_p.value(0)

    if dificultad == 3:
        led_p2.value(0)

    time.sleep(0.5)

    if rPositiva == False and rNegativa == False: #Analiza si no se agrego un punto en el juego anterior
        strikes.append(0)
        cajaMusical(5, 0.1)
        buzzer.duty_u16(0)
        rNegativa = True
        print("Strike")

    if strikes.count(0) == 3: #Si se cumple la cantidad de strikes, se rompe la iteracion
        print("break")
        break

    if (puntos == 5 and dificultad == 1) or (puntos == 7 and dificultad == 2): #Actualizar permisos de dificultad
        if permisos.count(2) == 0:
            permisos.append(2)

        elif permisos.count(3) == 0 and permisos.count(2) == 1:
            permisos.append(3)

    metronomo = 0
    interruptorInf(conglomeradoLeds,0)

    inicioCiclo = time.ticks_ms()
    tiempoActual = time.ticks_ms()
    while (time.ticks_diff(tiempoActual, inicioCiclo) < 3000):
        tiempoActual = time.ticks_ms()
        if metronomo== 1:
            interruptorInf(conglomeradoLeds,1)

            time.sleep(0.4)
            metronomo = 0
            numSegmento(puntos)

        else:
            interruptorInf(conglomeradoLeds,0)
            time.sleep(0.4)
            metronomo = 1
            apagarLeds(segLeds)

    puntos = 0
    dificultad = 1
    n = 0

```

Conclusion:

Este sistema demuestra que es posible crear una maquina capaz de interactuar, obtener información y realizar cálculos con los mismos para cumplir con una función previamente establecida. Es posible la comunicacion entre el sistema y el usuario solo con la adición de un par de switches y botones, permitiendo así que se pueda transmitir los deseos o necesidades del usuario en un rango previamente conocido.

Bibliografia:

Biblioteca estandar de python: <https://docs.python.org/es/3.10/library>

W3School; Python tutorial: <https://docs.python.org/es/3.10/library>

Curso Alura para el lenguaje de python: -